Your best bet is to avoid these and just use `filter_var()` / `filter_var_array()`, which are the two we'll focus on here.

FILTER_VALIDATE_*

The `filter_var*()` functions don't do anything without being passed at least one constant in addition to the data under scrutiny. There's a plethora of `FILTER_VALIDATE_*` and `FILTER_SANITIZE_*` constants, which control the functions' basic behavior, which can often be fine-tuned with additional `FILTER_FLAG_*` arguments (warning: some flag constants deviate from this naming convention). In this way the `filter` module avoids accumulating dozens of functions, unlike, say, the core string functions.

Table 11-43. *PHP FILTER_VALIDATE constants*

Validation Constants	Logic
FILTER_VALIDATE_INT	Is integer or valid string representation
FILTER_VALIDATE_FLOAT	Is float or valid string
FILTER_VALIDATE_BOOL	See below!
FILTER_VALIDATE_URL	Validate string per RFC 2396 (obsolete)
FILTER_VALIDATE_EMAIL	Per RFC 822 (also obsolete)
FILTER_VALIDATE_IP	Is IP address (v4 and v6 supported)
FILTER_VALIDATE_MAC	Is MAC address
FILTER_VALIDATE_REGEXP	Validate using regex

So, we have two general types of behavior: validating and sanitizing. Presumably validating operations return a boolean to indicate if the supplied data met some criteria or other, and sanitizing will return transformed input data so that it meets a certain set of criteria? Well, let's see some of this in action:

```
var_dump(filter_var(1234, FILTER_VALIDATE_INT));
var_dump(filter_var('1234', FILTER_VALIDATE_INT));
var_dump(filter_var('123.4', FILTER_VALIDATE_INT));
var_dump(filter_var('123.4', FILTER_VALIDATE_FLOAT));
```